

Specification Models

System Level Design and Modeling

II Semester, 2025-2026

Preeti Ranjan Panda

Department of Computer Science and Engineering

Indian Institute of Technology Delhi

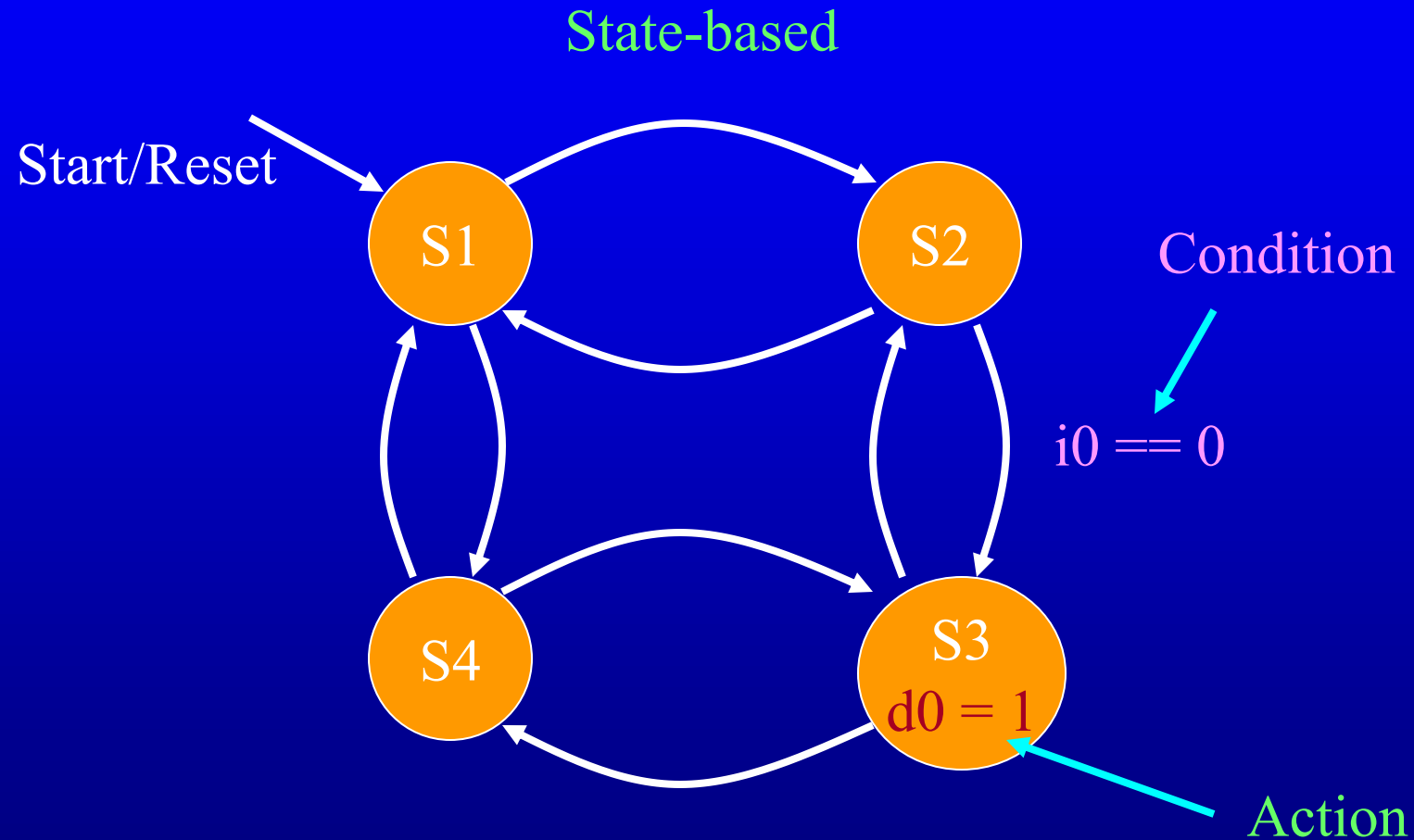
Specification Models

- State-oriented
- Activity-oriented
- Heterogeneous

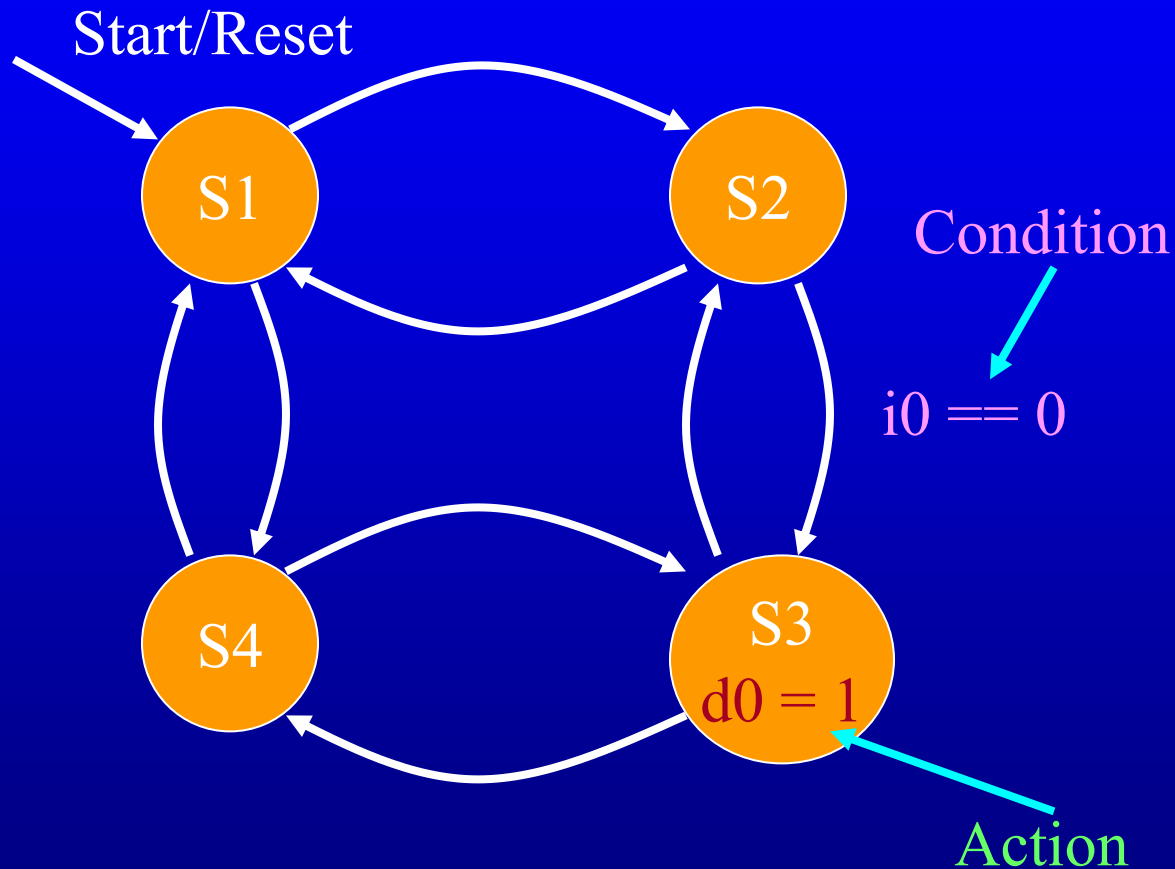
State-oriented models

- FSM (Finite State Machine)
- FSMD (FSM with Datapath)
- HCFSM (Hierarchical Concurrent FSM)

Finite State Machines: Moore Type



Moore FSM: Definition



Quintuple

$\langle S, I, O, f : S \times I \rightarrow S, h : S \rightarrow O \rangle$

S: Set of States $\{s_1, s_2, \dots, s_n\}$

I: Set of Inputs $\{i_1, i_2, \dots, i_n\}$

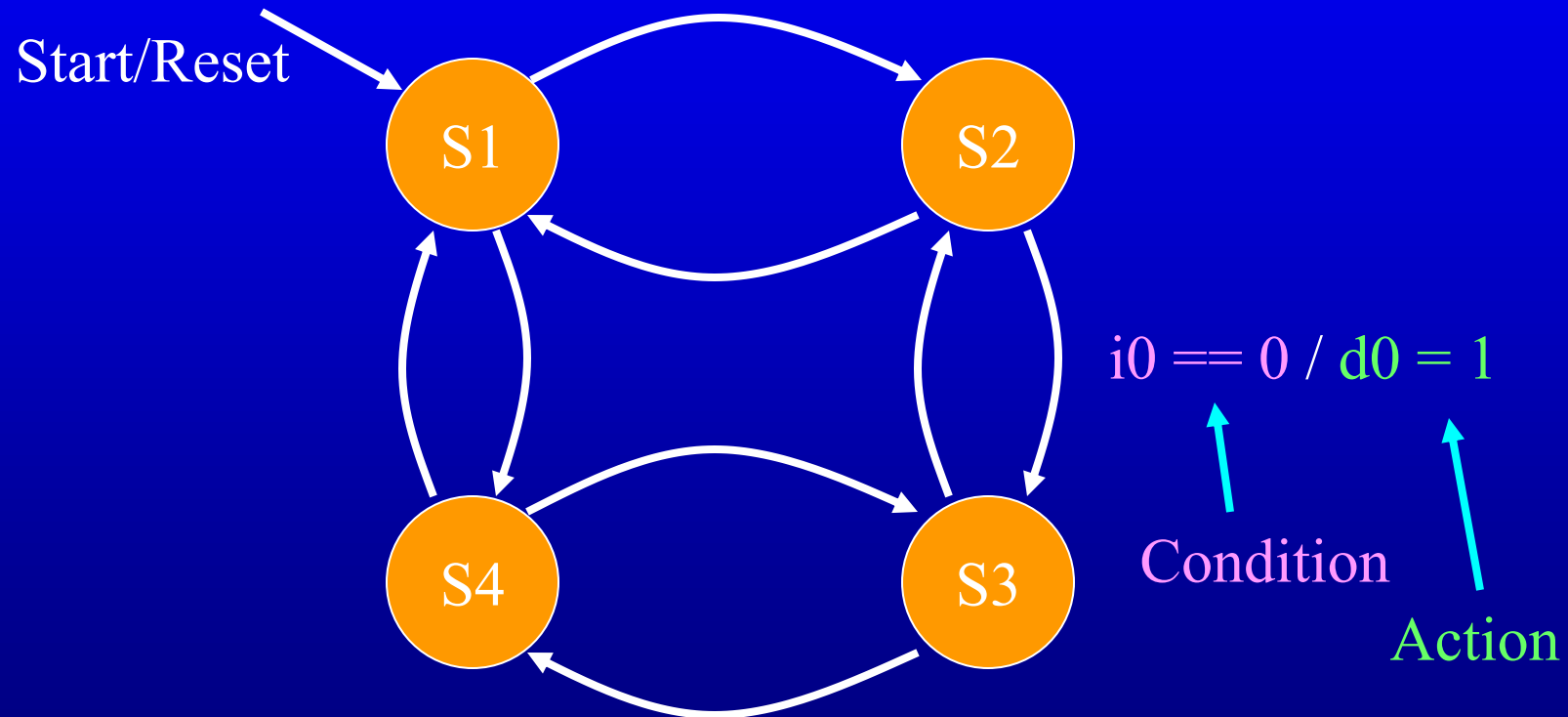
O: Set of Outputs $\{o_1, o_2, \dots, o_n\}$

f: Next-State Function

h: Output function

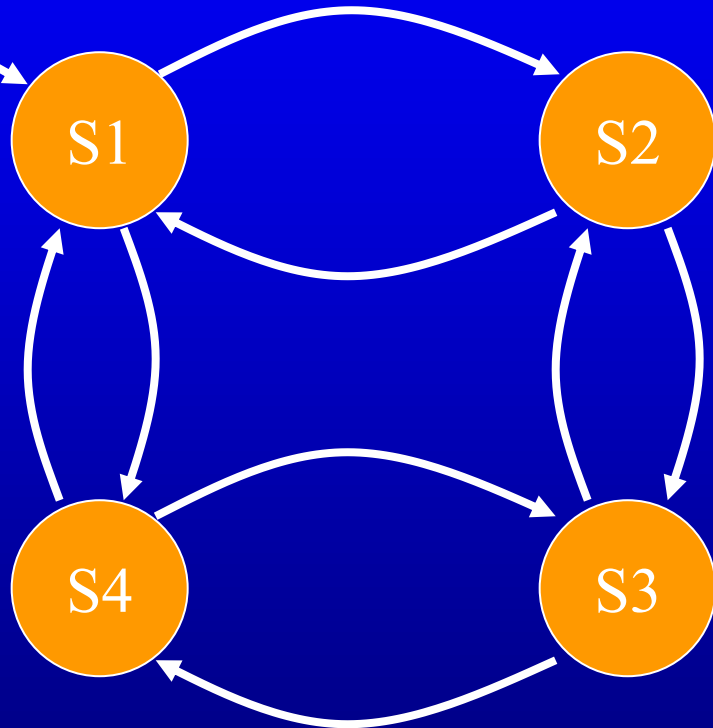
Finite State Machines: Mealy Type

Transition-based



Mealy FSM: Definition

Start/Reset



Quintuple

$\langle \mathbf{S}, \mathbf{I}, \mathbf{O}, \mathbf{f} : \mathbf{S} \times \mathbf{I} \rightarrow \mathbf{S}, \mathbf{h} : \mathbf{S} \times \mathbf{I} \rightarrow \mathbf{O} \rangle$

$i0 == 0 / d0 = 1$

Condition

Action

S: Set of States $\{s_1, s_2, \dots, s_n\}$

I: Set of Inputs $\{i_1, i_2, \dots, i_n\}$

O: Set of Outputs $\{o_1, o_2, \dots, o_n\}$

f: Next-State Function

h: Output function

Limitation of FSMs

- State Explosion
 - when representing data
 - 32-bit integer requires 2^{32} states

FSM with Datapath (FSMD)

Augment FSM with

- set of storage variables

$$\text{VAR} = \{v_1, v_2, \dots, v_n\}$$

- set of expressions

$$\text{EXP} = \{f(x, y, \dots) \mid x, y, \in \text{VAR}\}$$

- set of storage assignments

$$A = \{X \leftarrow e \mid X \in \text{VAR}, e \in \text{EXP}\}$$

- set of status expressions

$$\text{STAT} = \{\text{Rel}(a, b, \dots) \mid a, b \in \text{EXP}\}$$

Rel is any Relation, e.g., “<”

FSMD: Definition

Quintuple

$$\langle \mathbf{S}, \mathbf{I} \cup \mathbf{STAT}, \mathbf{O} \cup \mathbf{A}, \\ \mathbf{f} : \mathbf{S} \times (\mathbf{I} \cup \mathbf{STAT}) \rightarrow \mathbf{S}, \\ \mathbf{h} : \mathbf{S} \times (\mathbf{I} \cup \mathbf{STAT}) \rightarrow (\mathbf{O} \cup \mathbf{A}) \rangle$$

S: Set of States $\{s_1, s_2, \dots, s_n\}$

I $\cup$ **STAT**: Set of Inputs $\{i_1, i_2, \dots, i_n\}$ and Status Expressions

O $\cup$ **A**: Set of Outputs $\{o_1, o_2, \dots, o_n\}$ and Storage Assignments

f: Next-State Function uses Status Expressions also

h: Output function uses Status Expressions and also generates Storage Assignments

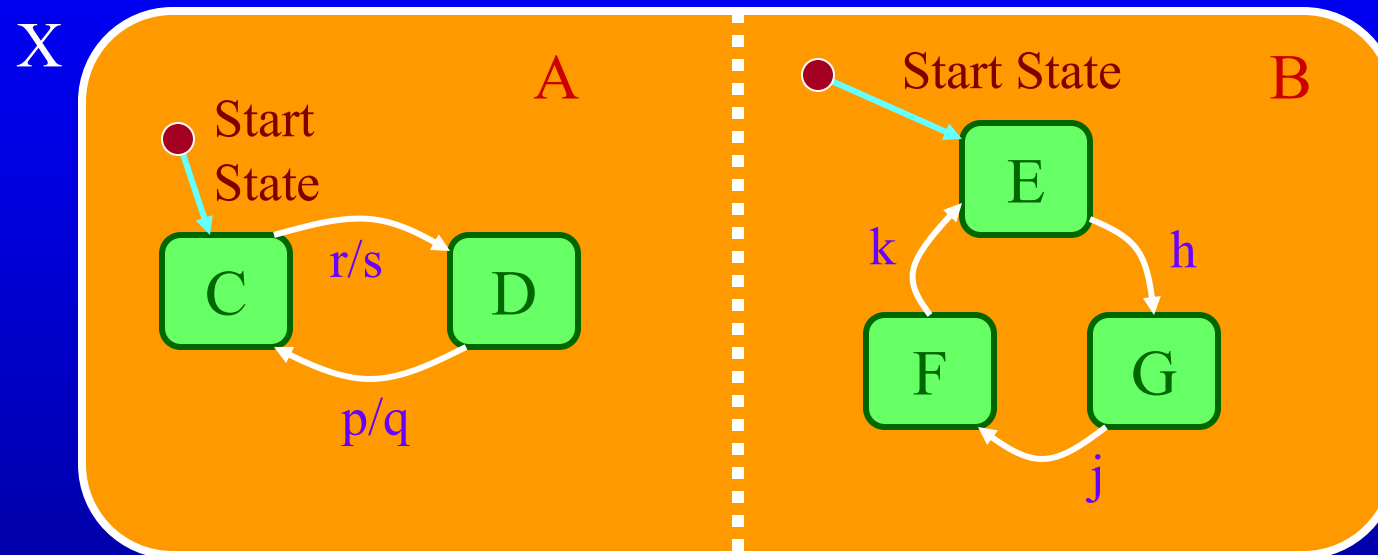
FSMD: Advantages

- No need for separate state for each integer value
- Can be used for both control-oriented and computation-oriented systems

Hierarchical Concurrent FSM (HCFSM)

- Extension of FSM model
 - Hierarchy
 - State can be decomposed into SUB-STATES
 - Concurrency
 - Multiple states can be active in parallel
 - Transitions
 - Structured: between states at same level
 - Unstructured: between states across hierarchy boundaries

HCFSM Example: Statecharts



Hierarchy: A, B are sub-states of X

C, D are sub-states of A

E, F, G are sub-states of B

Concurrency: A, B are concurrent

HCFSM: Features

- Good for complex control-dominated systems
- Weakness: actions are too simple

Activity-oriented models

- Data flow graph
- Flow chart

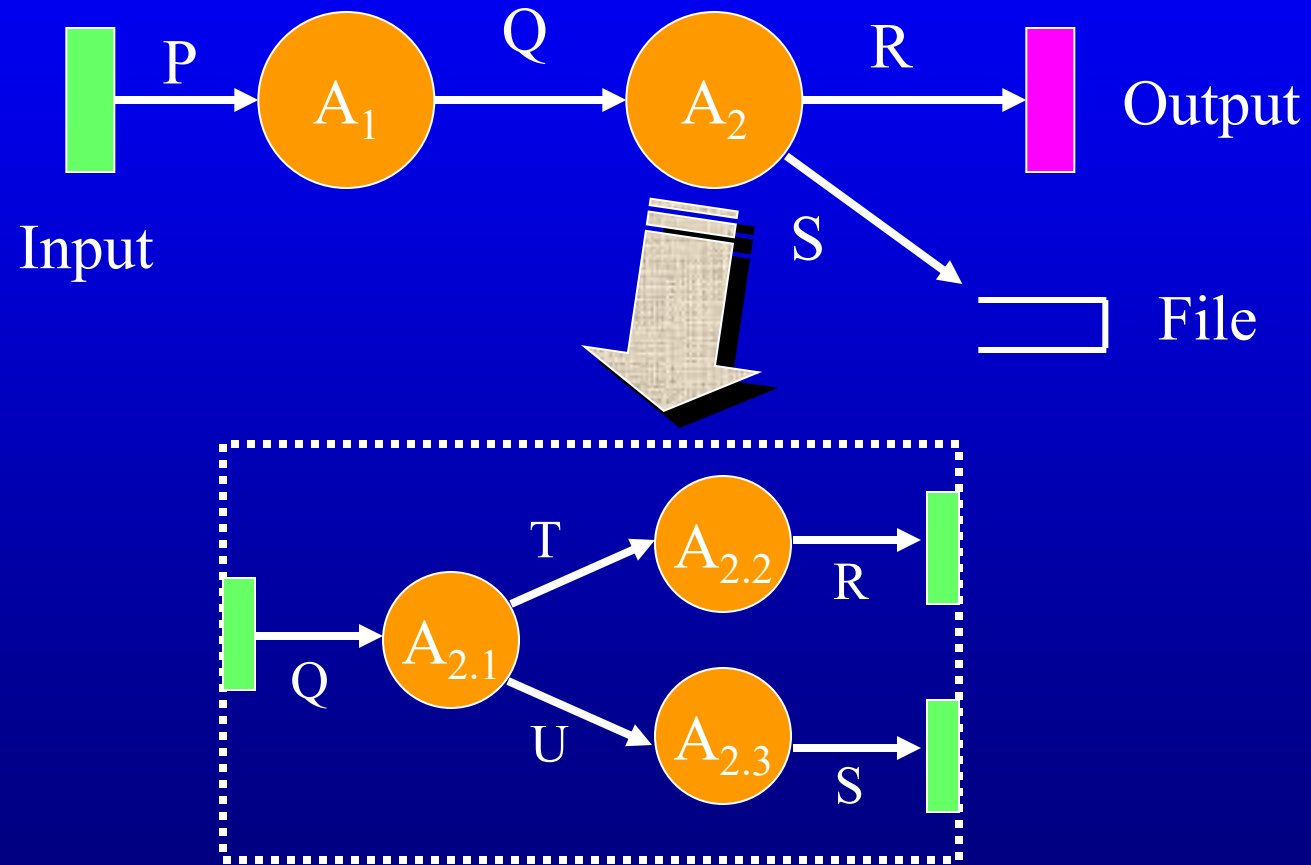
Data Flow Graphs

- Outputs determined by set of computations on inputs
 - no explicit notion of STATE
 - no external events trigger state changes
- Set of ACTIVITIES (or computations or transformations)
- Connected by set of arcs representing DATA FLOW

Data Flow Graphs: Definition

- Set of nodes
 - INPUT (or source) OUTPUT (or destination)
 - ACTIVITY (or process)
 - transformation and manipulation of data
 - different levels of granularity: program, function, instruction, operation
 - DATA STORE
 - variables in memory, file in disk, etc.
- Set of directed edges
 - labelled with DATA transmitted between i/o, activity, and storage nodes

Data Flow Graphs: Example



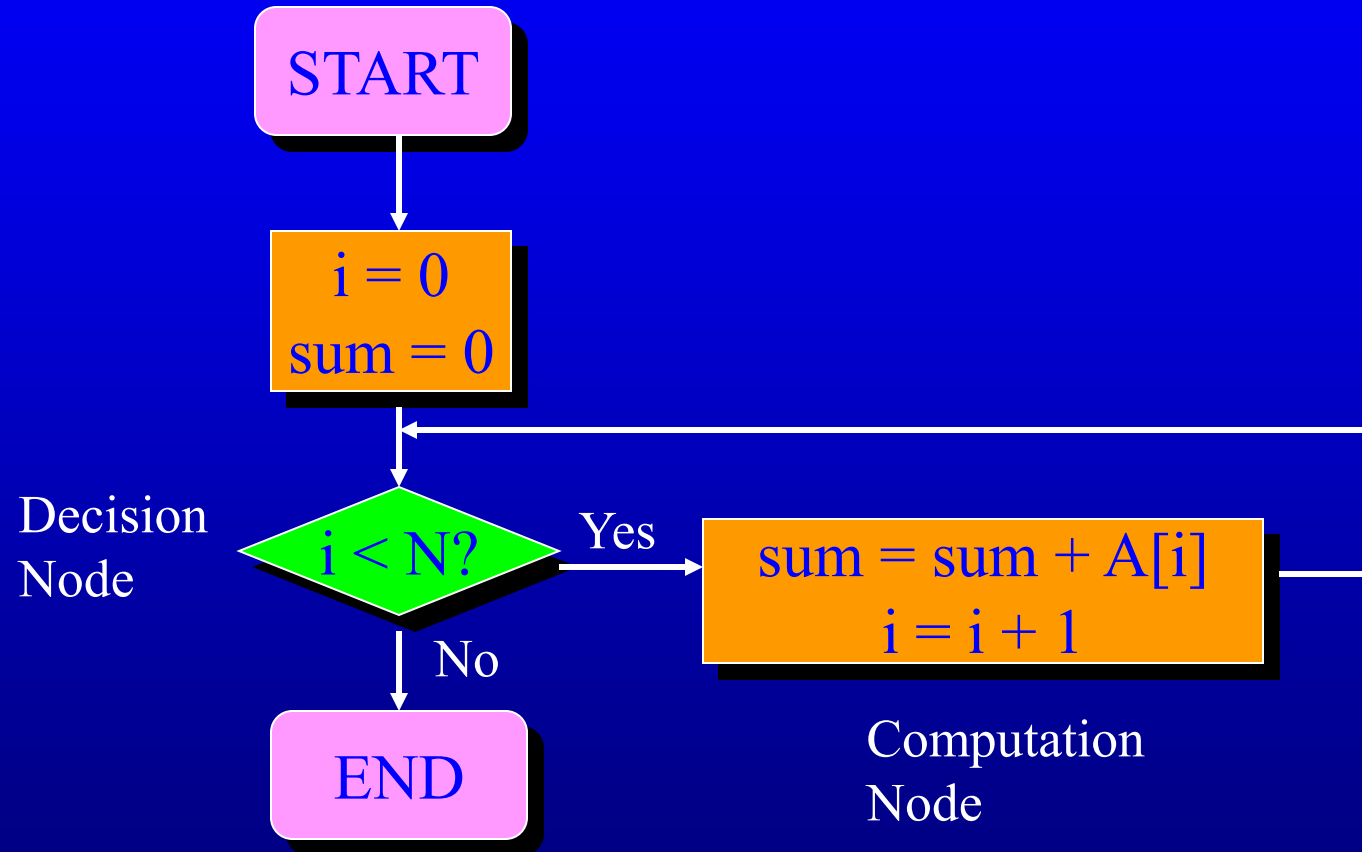
Data Flow Graphs: Features

- Can represent wide variety of systems
 - Level of abstraction of nodes and edges can vary
- No sequencing imposed
 - only data dependencies
- No implementation implied
- Hierarchy supported
 - an activity node can represent another DFG
- No temporal behavior
 - hard to model complex protocols

Flow Charts

- Set of nodes
 - START and END nodes
 - COMPUTATION node for data transformations
 - DECISION node for control flow/branching
- Directed edges
 - temporal order

Flow Charts: Example



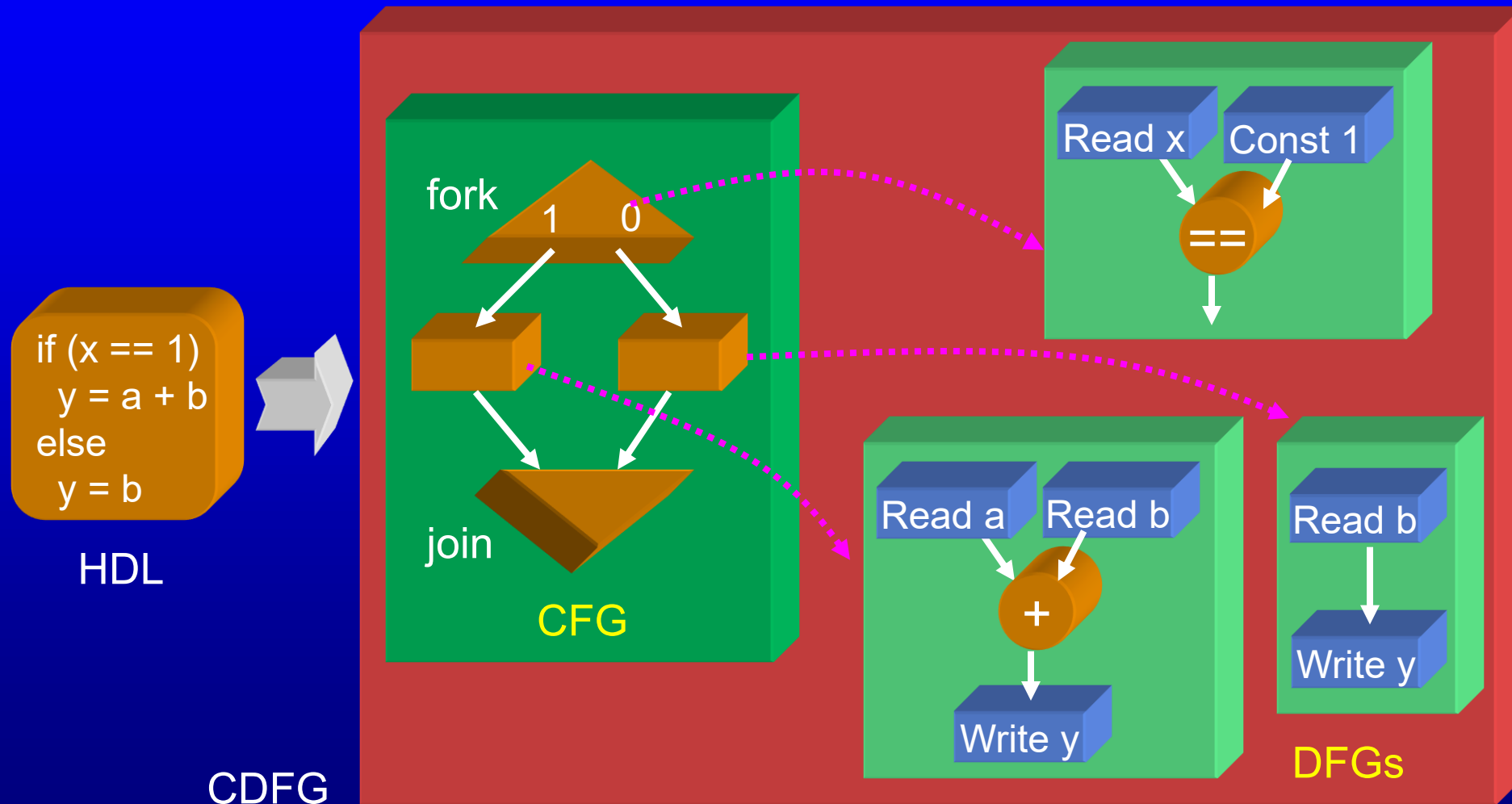
Flow Charts: features

- Similar to Data Flow Graphs
 - But arcs represent Control Dependencies
- Similar to FSMD
 - But transitions occur when activity completes
 - in FSMD, external events cause transitions
- Sequential Activities
- No dependence on external events

Heterogeneous models

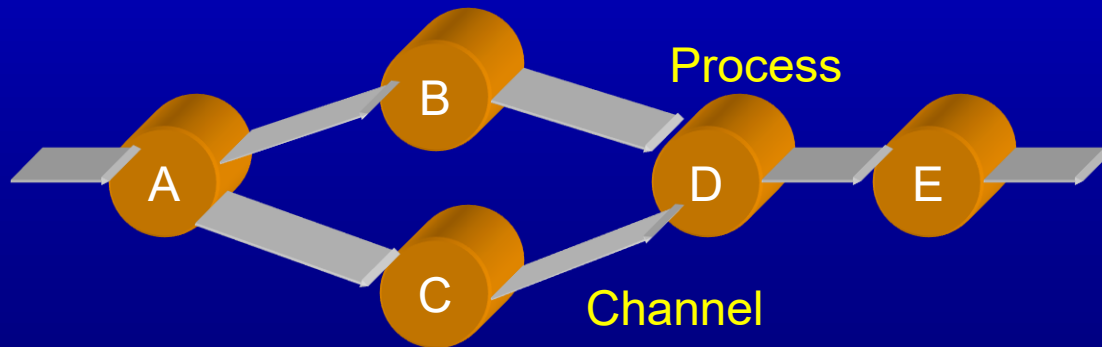
- **CDFG (Control Data Flow Graph)**
- **Process Networks**

Control Data Flow Graph (CDFG)



Process Networks

- Network of Processes (computation - nodes) connected by Channels (communication - edges: ordered data/token sequences)
 - Constrained subsets have useful properties
 - Modelling stream-based applications



Internal computation
specified in host language
(any programming language)

Network specified in
coordination language

Kahn Process Networks (KPN)

- Process communication occurs by sending data tokens on channels
- Channels are unidirectional FIFOs with unbounded capacity
- Non-blocking WRITE
 - writing into channel always succeeds
 - channel is never full
- Blocking READ
 - process suspends if not enough tokens on channel
 - channel can be empty

Sequences

Sequence $X = [a, b, c]$ Sequence $Y = [a, b, c, d]$
 $X \sqsubseteq Y$ (X is a prefix of - or equals - Y)

Empty sequence is prefix of all sequences $\perp \sqsubseteq X$

Consider increasing chain of sequences $\chi = \{X_0, X_1, X_2, \dots\}$
Where $X_0 \sqsubseteq X_1 \sqsubseteq X_2 \dots$

$X_0 = \{A, B\}$

$X_1 = \{A, B\}$

$X_2 = \{A, B, C\}$

$X_3 = \{A, B, C\}$

$X_4 = \{A, B, C\}$

p-tuples of Sequences

S^p = set of p-tuples of sequences

$$\mathbf{X} = \{X_1, X_2, \dots, X_p\} \in S^p$$

E.g., $\mathbf{X} = \{ \{a, b\}, \{q\}, \{r, s, t\} \}$ ($p = 3$)

Sets of sequences can be ordered:

$\mathbf{X} \subseteq \mathbf{X}'$ if $\mathbf{X}_i \subseteq \mathbf{X}'_i$ for all $1 \leq i \leq p$

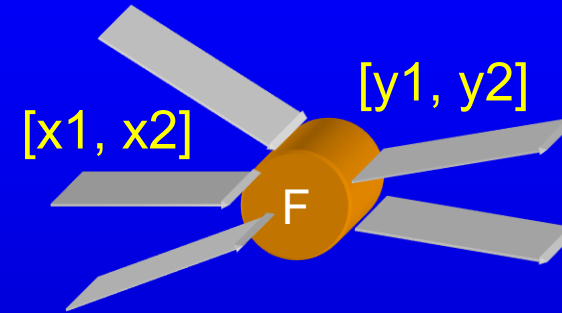
E.g., $\mathbf{X}' = \{ \{a, b\}, \{q, s\}, \{r, s, t, u\} \}$

Set L of p-tuples of sequences $L = \{\mathbf{X}_1, \mathbf{X}_2, \dots\}$

$\mathbf{X}_1 = \{ \{a, b\}, \{c\}, \{d, e, f\} \}$, $\mathbf{X}_2 = \{ \{a, b, r\}, \{c\}, \{d, e, g\} \}$

Processes

A Functional Process defines a mapping F from a set of input sequences X to a set of output sequences $Y = F(X)$



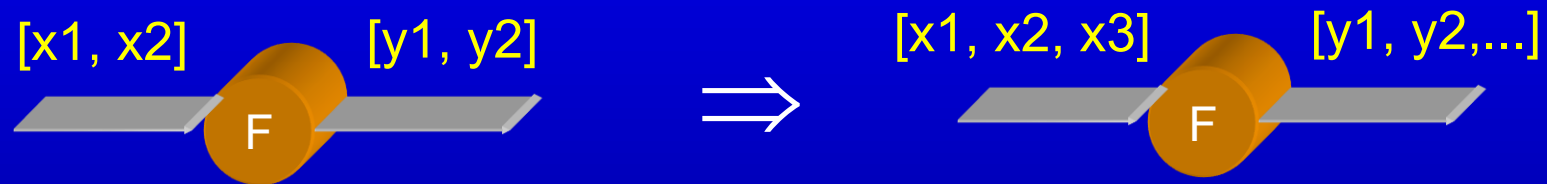
Given increasing chain of sets of sequences L
 F is CONTINUOUS if $\text{lub } (F(L))$ exists and (omitting discussion of **lub**)
 $\text{lub } (F(L)) = F(\text{lub } (L))$

This is similar to continuity in real functions: $y = f(x_0)$ exists and
 $\lim_{x \rightarrow x_0} f(x) = f(x_0)$

Informally, if you add one more token to the i/p stream, the o/p stream does not change drastically (in our case, o/p stream is also increasing)

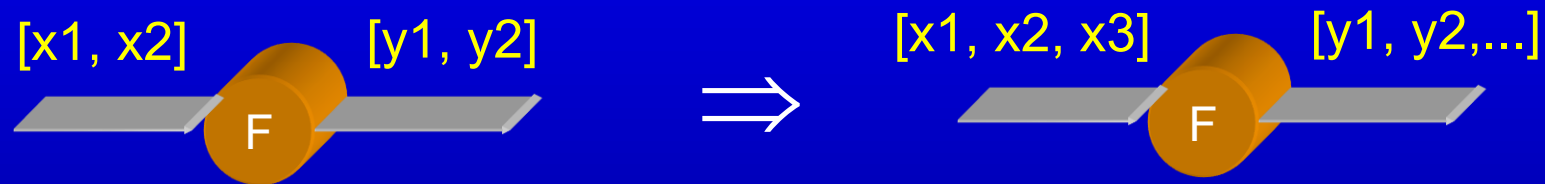
Monotonicity

Process is monotonic if $X \sqsubseteq X' \Rightarrow F(X) \sqsubseteq F(X')$



KPNs are Monotonic

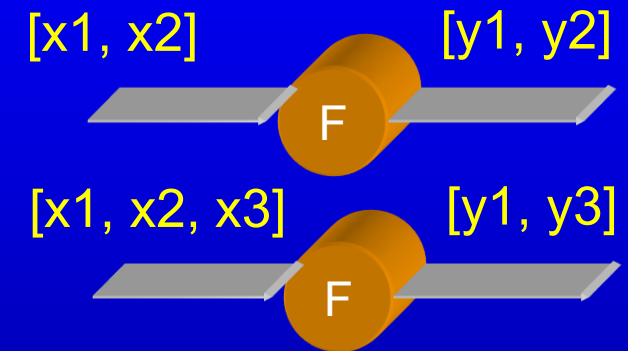
Kahn Processes are constrained to be CONTINUOUS
which implies that they are MONOTONIC (why? omitting)



X is, in general, a **set of sequences**, one for each input. Prefix ordering for set of sequences defined as: $\{ [a, b], [p, q] \} \sqsubseteq \{ [a, b, c], [p, q] \}$

Why Monotonicity?

- Given part of input sequences, we can compute part of output sequences
 - need not wait for completion of input sequence before producing an output
- Network of monotonic processes also monotonic
- Monotonicity leads to determinism



Non-monotonic Process

Kahn Processes are Deterministic

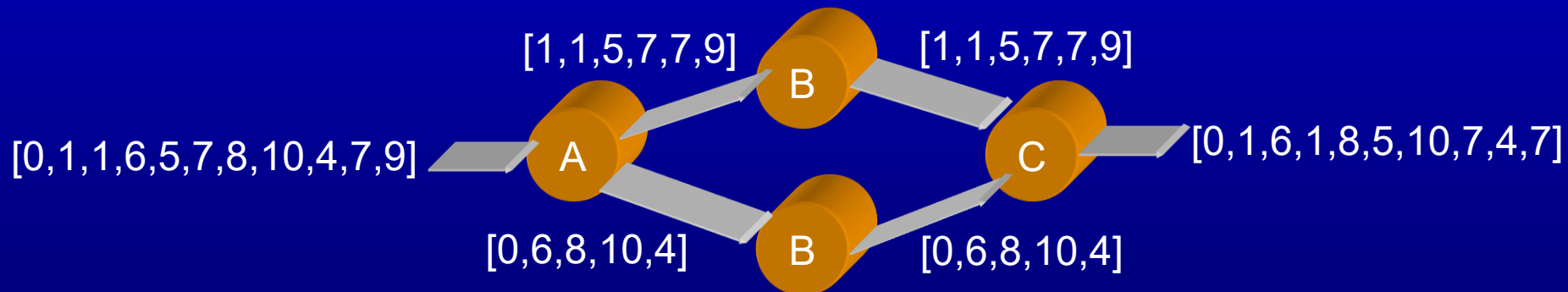
- For a given input sequence, there is only one output sequence
- Output is independent of rates at which individual processes execute

Example KPN

```
Process A (in int x, out int y,  
           out int z) {  
  loop  
    int k = x.read;  
    if k is even y.write (k);  
    else z.write (k);  
  end loop;  
}
```

```
Process B (in int x,  
           out int y) {  
  loop  
    int k = x.read;  
    y.write (k);  
  end loop;  
}
```

```
Process C (in int x, in int y,  
           out int z) {  
  int k; bool s = TRUE;  
  loop  
    if (s) {  
      k = x.read; z.write (k);  
    } else {  
      k = y.read; z.write (k);  
    }  
    s = NOT s;  
  end loop;  
}
```

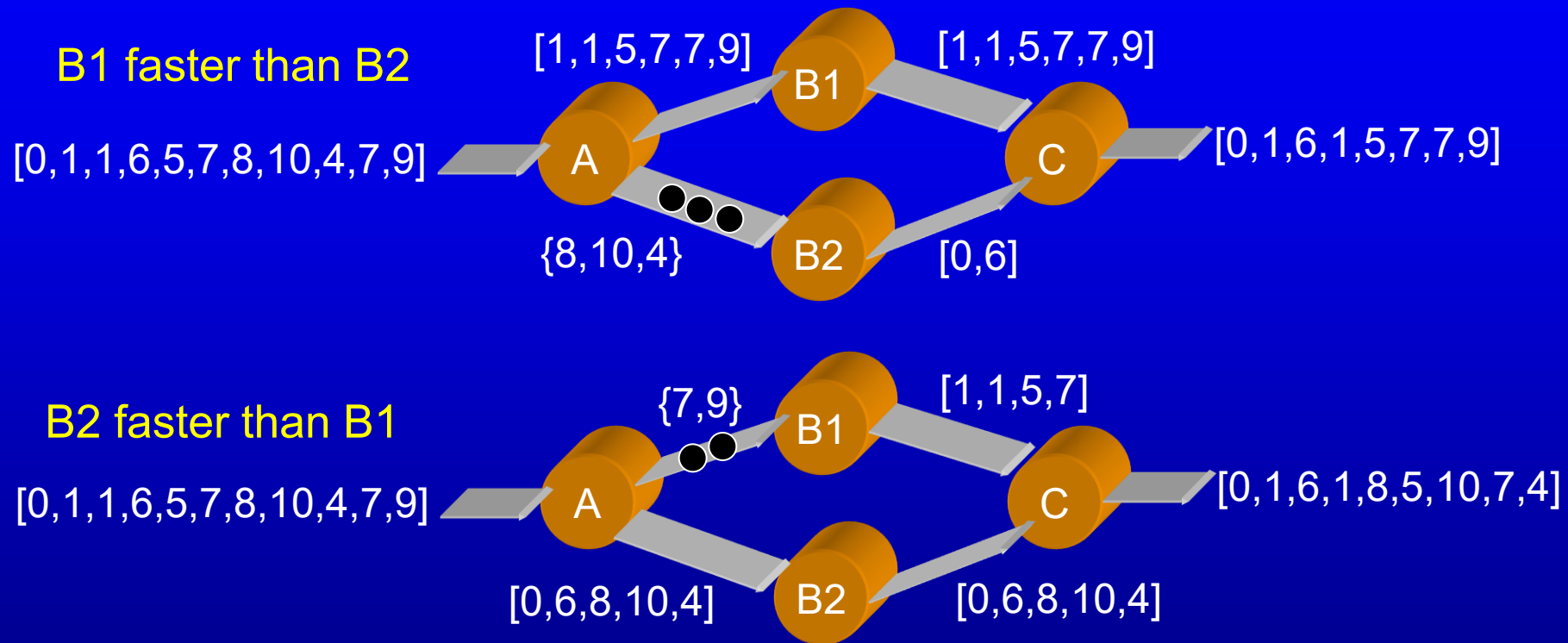


Checking Status of Channel

```
Process C (in int x, in int y, out int z) {  
  int k; bool s = TRUE;  
loop  
  if (s) {  
    k = x.read on timeout(10ns)  
    select on x,y  
      x: k = x.read;  
    or  
      y: k = y.read;  
  } else {  
    k = y.read; on timeout(10ns)  
    select on x,y  
      x: k = x.read;  
    or  
      y: k = y.read;  
  }  
  z.write (k); s = NOT s;  
end loop;  
}
```

- try reading from channel
- if no tokens for 10ns, read from channel where a token is first available

Non-determinism



Output sequence depends on relative speed of B1 and B2
Process network is **NON-DETERMINISTIC**

Introducing Non-determinism

- Test input channels for emptiness
- Internally non-determinate processes
- Multiple writer processes to a channel
- Multiple reader processes from a channel
- Shared variables between processes

Implementing KPN

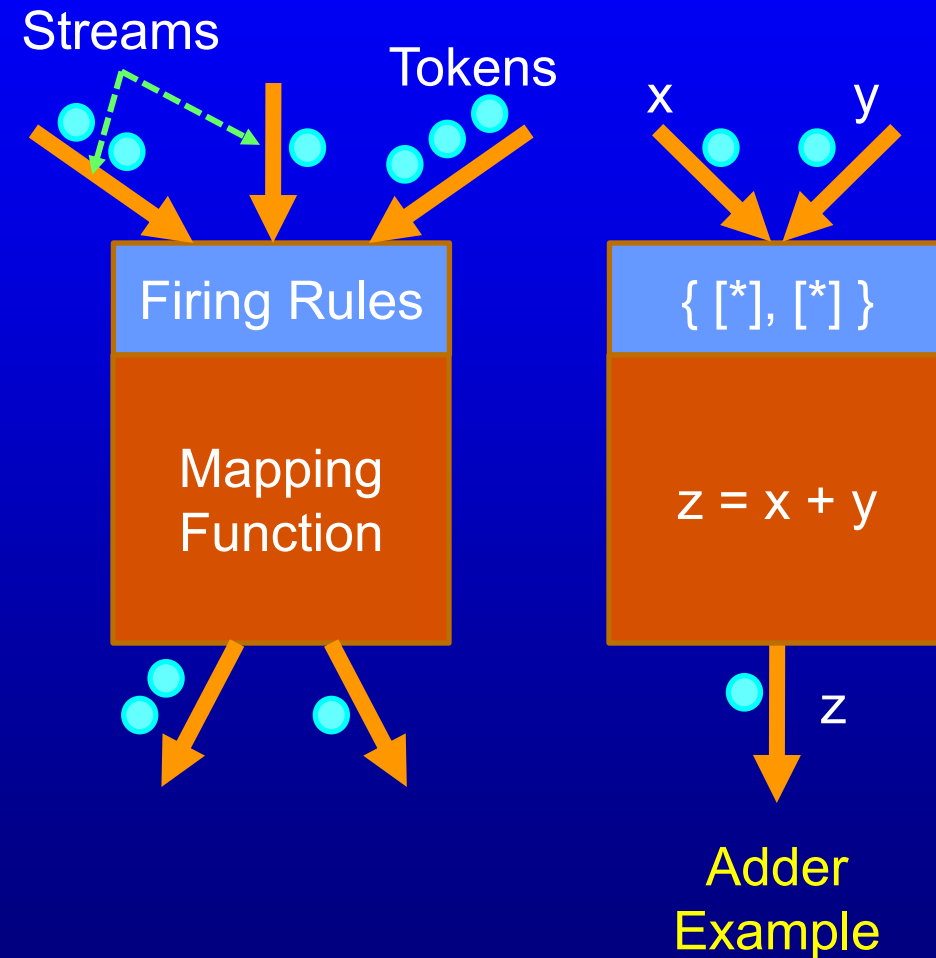
- Resource constraints: e.g., 1 processor
 - run-time scheduler needed
- Context switching overhead
 - processes need to be suspended and resumed
 - if fine-grained computation, too many context switches
- KPN is too powerful
 - simplification/restrictions needed

Dataflow Process Networks

- Actor maps input tokens to output tokens
 - firing (activation) consumes input tokens and produces output tokens
- Firing rules specify when an actor can fire
 - what input tokens should be available

Dataflow Actors

- Consist of:
 - Firing Rule
 - Mapping/Function

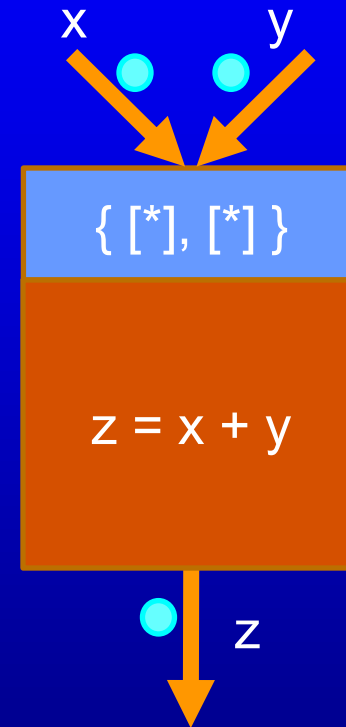


Dataflow Process Networks

- Continuity property holds if
 - firing is functional
 - o/p tokens are function only of i/p tokens used in that firing
 - no side-effects
 - firing rules are sequential
 - rules can be tested in a pre-determined order
 - ...using only BLOCKING READS

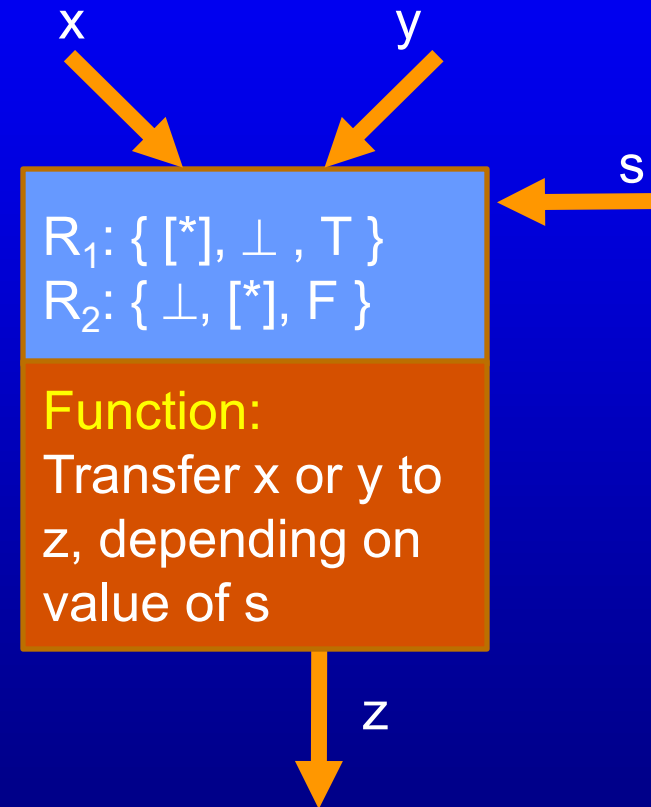
Adder Example

- Can ADDER be implemented with BLOCKING READS?



Select Example

- Can SELECT be implemented with BLOCKING READS?



Implementing Dataflow Process Networks

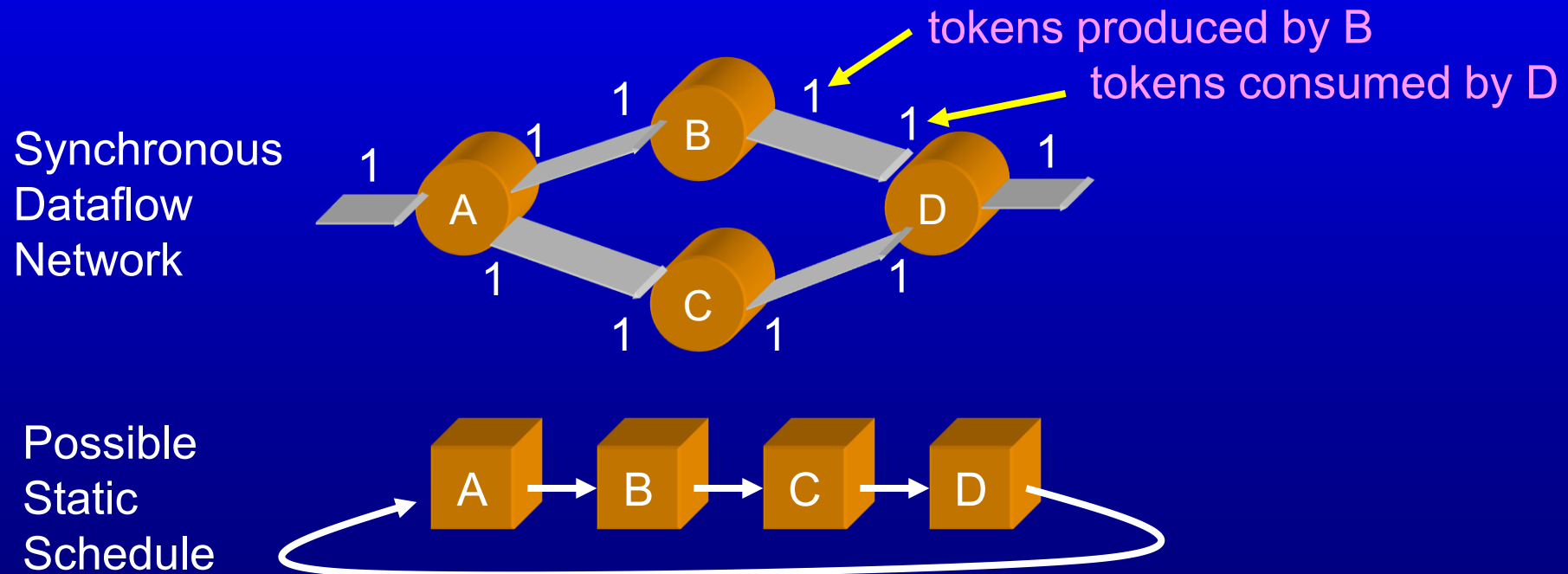
- Stronger restriction than KPN
 - actors do not have state
- Dynamic scheduler still needed
- No need of multiple processes
 - because actors are functional
 - no need of suspension
- Clusters of processes may be statically schedulable
 - by observing firing rule relationships

Synchronous Dataflow

- Static schedule
 - order of process activation fixed at compile-time
- Further constrain Dataflow Process Network
 - only one firing rule
 - firing consumes and produces fixed number of tokens on each i/p and o/p channel
 - all i/p tokens must be present to activate

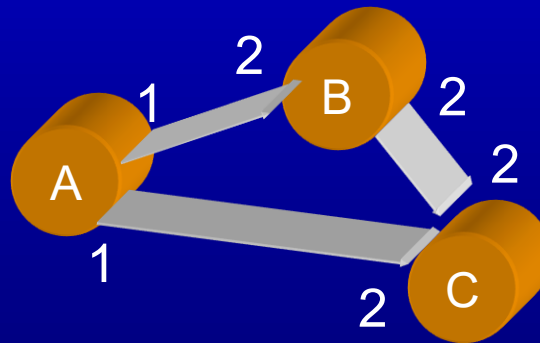
Example Static Schedule

- Each actor consumes and produces one token
- Periodic static schedule can be derived



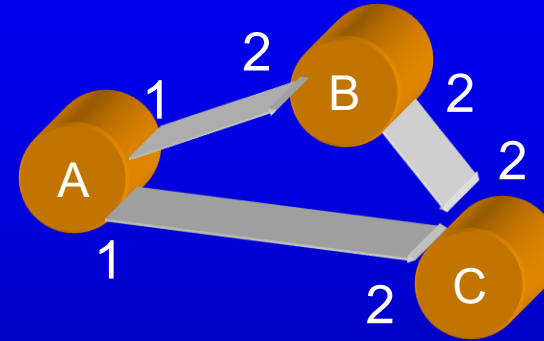
Deriving a Static Schedule

- Find actor firing sequence that returns the network to original state
 - this represents a static schedule that is cyclic
- Returning to original state ensures finite memory



Balance Equations

- Derive system of linear equations
 - one equation for each channel, one variable for each actor
 - no net change in # tokens at end of sequence
- Synchronous Dataflow Network if non-zero solution exists
- We need smallest positive integer solution
 - how many firings of actor in each cycle



Balance Equations

$$a - 2b = 0$$

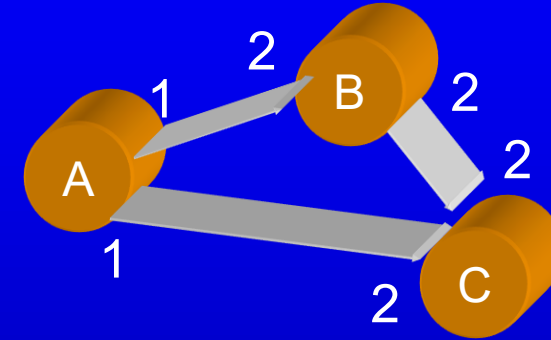
$$2b - 2c = 0$$

$$a - 2c = 0$$

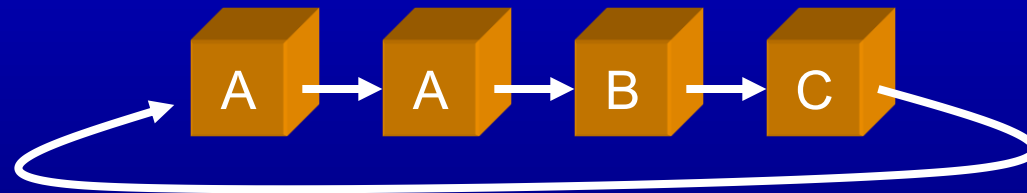
Solution: $a=2, b=1, c=1$

Periodic Static Schedule

- Other valid solutions exist
 - $a=4, b=2, c=2$
- If no solution, no static periodic cycle with finite memory



Solution: $a=2, b=1, c=1$



Static Schedule

Process Network Models

- Trade-off between expressive power and efficiency of implementation